

Code Review Summary Report

Overview

The purpose of this assignment was to improve a previously completed Boggle Solver project by participating in a professional-style code review process. The project required students to upload their Python Boggle Solver implementation to GitHub, review another student's code, receive peer feedback, and improve their own code using static analysis tools and refactoring techniques.

My approach to the assignment focused on creating a recursive depth-first search (DFS) algorithm capable of exploring all possible letter combinations on a Boggle board. The solver checks neighboring cells recursively in all eight directions while preventing revisiting the same cell within a single word path. Valid words are compared against a provided dictionary and stored in a set to avoid duplicates.

The second phase of the assignment emphasized software engineering best practices such as readability, maintainability, collaboration, and testing. I used peer feedback and linting tools to improve the overall quality of my code.

Feedback I Received

My reviewer stated that the code was functional and that the overall logic of the solver was correct. The feedback highlighted that the recursive search strategy worked properly and that the algorithm successfully identified valid words from the board.

Although the feedback was mostly positive, I recognized opportunities to further improve the readability and maintainability of the code. The review process encouraged me to think more critically about code clarity, naming conventions, formatting consistency, and documentation.

Feedback I Gave

While reviewing my partner's code, I focused on readability, organization, and maintainability. I looked for areas where comments could improve understanding and where variable names could be more descriptive. I also checked whether functions were too large or overly complex and suggested simplifying logic where possible.

In addition, I reviewed formatting consistency and overall code structure to ensure the program followed good programming practices and was easy to understand for future developers.

Improvements I Implemented

After receiving feedback, I refactored my Boggle Solver to improve readability and organization. One of the major improvements was adding descriptive comments throughout the code to explain the purpose of functions, recursive logic, and important steps in the algorithm.

I also renamed several variables to make them more meaningful and easier to understand. For example, shorter or unclear variable names were replaced with descriptive names such as `visited_cells`, `current_word`, and `found_words`. This made the program more readable and easier to maintain.

Additionally, I improved formatting and spacing throughout the file to better align with Python coding standards. I added proper blank lines, adjusted indentation, and organized the code structure to improve clarity.

Static Analysis Results

To improve code quality, I used the `pycodestyle` static analysis tool to check my program against PEP 8 style guidelines. Initially, the linter detected a formatting issue related to a missing newline at the end of the file.

After correcting the issue, I reran the linter and confirmed that the program passed the style checks successfully. Using static analysis helped reinforce the importance of writing clean and professionally formatted code.

Regression Testing

After making improvements to the code, I performed regression testing to ensure the program still functioned correctly. I reran the Boggle Solver and verified that valid words were still being discovered properly and that the recursive search algorithm continued to work as expected.

Regression testing was important because it confirmed that readability and formatting improvements did not introduce new bugs or break existing functionality.

Reflection

This assignment helped me better understand the importance of code reviews in professional software development. Reviewing another student's work gave me experience analyzing code quality, readability, and maintainability from a developer's perspective.

I also learned that code reviews are not only about finding errors but also about improving communication, organization, and collaboration within a development team. The process

showed me how even functional code can still be improved through better formatting, comments, and naming conventions.

Finally, using tools such as GitHub, Git branching, pull requests, and static analysis software provided valuable experience with workflows commonly used in real-world software engineering environments. This project strengthened both my technical coding skills and my understanding of collaborative development practices.